

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process started.\n");
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());

    printf("\nProcess is Running...\n");

    printf("\nProcess will enter Waiting state for 10 seconds.\n");
    sleep(10);

    printf("\nProcess is Running again.\n");

    printf("Press Enter to terminate the process...\n");
    getchar();

    printf("Process is Terminating...\n");

    return 0;
}
```

Output:

```
priyanka@DESKTOP-HOIS07A: /mnt/c/Users/DELL/OneDrive/Documents/S OS/Practicapriyanka@DESKTOP-HOIS07A: /mnt/c/Users/DELL/OneDrive/Documents/S OS/Practical/Week
3$ gcc task3.2.c -o task3.2
priyanka@DESKTOP-HOIS07A: /mnt/c/Users/DELL/OneDrive/Documents/S OS/Practical/Week 3$ ./task3.2
Process started.
PID: 1221
PPID: 337

Process is Running...

Process will enter Waiting state for 10 seconds.

Process is Running again.
Press Enter to terminate the process...
```

Observation: The **top** command showed 27 processes: 1 running, 26 sleeping, 0 stopped, and 0 zombie processes. The **R** state represents a running/runnable process, while **S** represents a sleeping/waiting process. Most processes were in the sleeping state because they were waiting for events or CPU time. No stopped or zombie processes were observed during the experiment.